AGENTIC RAG | PRODUCTION CLOUD GUIDE

Agentic RAG — Azure Deployment Guide

Service catalog and step-by-step production deployment

Application: Canonical Agentic RAG (FastAPI + React + Redis + ChromaDB)
Pipeline: canonical_pipeline_version = v1
Document date: 8 September 2026
Cloud: Azure

What this document covers

This guide maps the Agentic RAG repository onto a production-shaped Azure topology. It lists every cloud service you need, why it exists, and a concrete ordered procedure to build, configure, ingest, and verify the stack. It is written against the current code: FastAPI in `src/api/server.py`, the React/nginx frontend, Redis cache and rate limits, Chroma HTTP, and the production boot checks in `ENVIRONMENT=production`.

Recommended path on Azure is Azure Container Apps plus Azure Cache for Redis, Azure Files, and Key Vault. A single VM + Docker Compose box is documented as staging only.

Contents

1. What you are deploying

Agentic RAG is not a notebook. Production is a multi-container service: a FastAPI agent runtime, a React UI that reverse-proxies `/api` and injects `X-API-Key` server-side, Redis for answer cache / rate limits / idempotency, and a Chroma HTTP server for embeddings. Optional pieces are Prometheus/Grafana, LangSmith tracing, NVIDIA rerank, Groq LLM failover, and Supabase (or a managed Postgres) for conversation memory and thumbs-up/down feedback.

Workload	Image / runtime	Port	Notes
API	Repo `Dockerfile` (Python 3.10, uvicorn)	8000	Liveness `GET /health`. Readiness `/health/ready` is auth-gated. SSE on `POST /query/stream` (idle up to 300s).
Frontend	`frontend/Dockerfile` (nginx 1.27)	80	Proxies `/api/` to the API. Injects `API_KEY`. SSE buffering must stay off. Upload body 25 MB.
Redis	redis:7 (managed service in cloud)	6379	Required in production for shared rate limits and cache. Password/auth token mandatory.
Chroma	chromadb/chroma:0.6.3	8000	Set `CHROMA_MODE=http`. Persist the volume. Do not publish this port to the internet.
Docs / SQLite	Shared file volume	—	PDFs, parent store, extra-source SQLite, eval DBs. In-process BM25 lives in the API.

Production boot will refuse to start unless all of the following are true:

- `OPENAI_API_KEY` is set.
- `API_KEY` is set and at least 32 characters (`openssl rand -hex 32`).
- `CORS_ORIGINS` lists real frontend origins — `*` is rejected.
- `API_WORKERS > 1` only if `RATE_LIMIT_BACKEND=redis`.
- `TRUST_CLIENT_RBAC` stays `false`.

> **Scale-out constraint.** BM25, the semantic cache, and in-process ingest jobs are not shared across replicas. Keep `API_WORKERS=1` and a single API task until you move hybrid search to a managed vector store. Size `MAX_CONCURRENT_QUERIES` against your OpenAI concurrency, not against CPU.

External systems the app already calls (not provisioned in the VPC):

System	Used for	Required?
OpenAI	Chat completions + `text-embedding-3-small`	Yes
NVIDIA NIM (build.nvidia.com)	Cross-encoder rerank if `RERANK_PROVIDER=nvidia`	Recommended
Groq	LLM failover when OpenAI fails	Optional
LangSmith	Parent-span traces `agent_request:<mode>`	Optional
DuckDuckGo	Web fallback in the graph (circuit-broken)	Optional
Supabase	Persistent chat memory + answer_feedback	Optional; replace with managed Postgres

Environment variables to set on the API

Variable	Production value
ENVIRONMENT	production
OPENAI_API_KEY	from secrets manager / Key Vault
API_KEY	same 32+ char secret injected into frontend nginx
REQUIRE_API_KEY	true
CORS_ORIGINS	https://rag.example.com
TRUSTED_HOSTS	rag.example.com
TRUST_PROXY_HEADERS	true (TLS terminates at the load balancer)
CHROMA_MODE	http
CHROMA_HOST / CHROMA_PORT	/ 8000
REDIS_URL	rediss://:${REDIS_KEY}@redis-rag-prod.redis.cache.windows.net:6380/0
CACHE_ENABLED	true
RATE_LIMIT_BACKEND	redis
API_WORKERS	1
PROTECT_METRICS_ENDPOINT	true unless scraper is on a private network
PROTECT_READINESS_ENDPOINT	true — load balancer probes `/health` only
NVIDIA_API_KEY	if rerank stays on NVIDIA
GROQ_API_KEY	optional failover
INGEST_ALLOWED_ROOTS	/app/data
REQUEST_TIMEOUT_SECONDS	120
STREAM_TIMEOUT_SECONDS	300
MAX_CONCURRENT_QUERIES	8 (tune after a load test)

Frontend container needs `API_UPSTREAM` (private API URL) and the **same** `API_KEY`. The browser never holds the key; nginx injects `X-API-Key` and strips any client-supplied one.

2. Complete Azure service catalog

2.1 Required services

Azure service	Role in this app	Maps to compose
Resource group	Single lifecycle for rag-prod	project folder
Azure Virtual Network	Private API/Redis/Chroma; public ingress only	compose network
NAT Gateway	Outbound to OpenAI / NVIDIA / Groq without public IPs on apps	host egress
Network security groups	Allow 443 in; 80/8000/6379 only on app subnets	expose vs ports
Azure Container Registry	API and frontend images	docker build
Azure Container Apps Environment	Managed serverless Kubernetes under the hood	compose project
Container App: frontend	nginx UI, external ingress	frontend :8080
Container App: api	FastAPI, internal ingress only	agentic-rag :8000
Container App: chroma	Chroma HTTP, internal ingress	chroma
Azure Cache for Redis	Cache, rate limits, idempotency	redis
Azure Files (Premium)	Chroma data + /app/data PDFs and SQLite	named volumes
Azure Blob Storage	Source corpus and backups	data/sample_docs
Azure Key Vault	OPENAI_API_KEY, API_KEY, Redis key, NVIDIA, Groq	.env.production
Managed identities	Apps pull secrets and mount storage — no keys in env files	IAM task role
Azure DNS / App hostname	https://rag.example.com on the frontend ingress	local :8080
Azure Monitor + Log Analytics	stdout JSON logs, container metrics	compose logs

2.2 Strongly recommended

Azure service	Why
Azure Front Door (Standard/Premium) + WAF	Global HTTPS, WAF, and optional CDN in front of Container Apps
Azure Application Gateway	Alternative regional L7 + WAF if you skip Front Door
Azure Database for PostgreSQL Flexible Server	Replace Supabase for chat_messages and answer_feedback
Azure Monitor managed Prometheus + Azure Managed Grafana	Scrape /metrics on the internal API
Microsoft Defender for Cloud	Registry scanning and runtime recommendations
Azure Policy	Deny public Redis, require HTTPS, require diagnostic settings
Azure Budgets	LLM spend plus Container Apps + Redis
Azure Backup / Blob lifecycle	Retain corpus and file-share snapshots

2.3 Optional / later

Azure service	When to add it
Azure Kubernetes Service (AKS)	When you outgrow Container Apps (custom CNI, DaemonSets, GPU nodes)
Azure AI Search / Azure OpenAI	When you replace Chroma+OpenAI embeddings with Azure-native RAG
Azure Service Bus	When ingest jobs must survive API restarts (today the queue is in-process)
Azure Container Apps jobs	One-off ingest and eval_gates instead of exec into the API
Azure App Service	Not a good fit: SSE, sidecar Redis/Chroma, and custom nginx key injection
Azure VM + docker compose	Cheap staging; same as local PRODUCTION.md

3. Recommended topology

Use **Azure Container Apps** on a VNet-injected environment. Frontend ingress is external (or Front Door → internal). API and Chroma ingress are internal. Azure Cache for Redis is VNet-injected. Azure Files mounts provide durable Chroma and PDF storage. Key Vault plus user-assigned managed identities replace `.env.production` files.

```
Internet
   ■  HTTPS  (Front Door + WAF, or Container Apps built-in cert)
   ▼
Container App  frontend  (external)
   ■  API_UPSTREAM=http://api.internal.<env>.azurecontainerapps.io
   ■  API_KEY from Key Vault reference
   ▼
Container App  api  (internal only)   ■■ NAT ■■■ OpenAI / NVIDIA / Groq / LangSmith
   ■
   ■■■ Azure Cache for Redis  (SSL, access key or Entra auth)
   ■■■ Container App  chroma  (internal) + Azure Files /chroma/chroma
   ■■■ Azure Files  /app/data  (PDFs, parent_store.json, knowledge.db)

Storage account   corpus container (PDFs)
Key Vault         agentic-rag-prod
Log Analytics     workspace rag-prod
```

Container Apps ingress idle timeout and HTTP/2 streaming: set `scale.rules` so the API replica is not scaled to zero, and raise the ingress timeout to cover `STREAM_TIMEOUT_SECONDS=300`. A scale-to-zero API will make the first chat look like an outage.

4. Prerequisites

- Azure subscription with Owner or equivalent on a dedicated resource group.
- Azure CLI 2.x, Docker, this repo. Login: `az login`.
- DNS name you control, or accept the azurecontainerapps.io hostname for staging.
- OpenAI key; NVIDIA key if rerank stays NVIDIA; 32+ character app API key.
- Quota: Container Apps, Redis, Files Premium in the target region (example: eastus).

```
export LOC=eastus
export RG=rg-agentic-rag-prod
export ACR=acragenticrag$RANDOM
export ENV=cae-agentic-rag
export KV=kv-agentic-rag-prod
export APP_HOST=rag.example.com
export API_KEY=$(openssl rand -hex 32)
az account show
```

5. Step-by-step deployment

Step 1 — Resource group and network

```
az group create -n $RG -l $LOC

az network vnet create -g $RG -n vnet-rag --address-prefix 10.20.0.0/16 \
  --subnet-name snet-apps --subnet-prefix 10.20.1.0/24

az network vnet subnet create -g $RG --vnet-name vnet-rag -n snet-redis \
  --address-prefix 10.20.2.0/24

az network vnet subnet create -g $RG --vnet-name vnet-rag -n snet-storage \
  --address-prefix 10.20.3.0/24

az network nat gateway create -g $RG -n nat-rag -l $LOC
az network public-ip create -g $RG -n pip-nat --sku Standard --allocation-method Static
az network nat gateway public-ip associate -g $RG -n nat-rag --public-ip-address pip-nat
az network vnet subnet update -g $RG --vnet-name vnet-rag -n snet-apps --nat-gateway nat-rag
```

Delegation: the apps subnet must be delegated to `Microsoft.App/environments` when you create the Container Apps environment with VNet injection. Redis subnet should use a private endpoint or VNet injection for Azure Cache for Redis (Premium).

Step 2 — Azure Container Registry and images

```
az acr create -g $RG -n $ACR -l $LOC --sku Standard --admin-enabled false
az acr login -n $ACR

docker build -t $ACR.azurecr.io/agentic-rag-api:v1 .
docker build -t $ACR.azurecr.io/agentic-rag-frontend:v1 ./frontend
docker push $ACR.azurecr.io/agentic-rag-api:v1
docker push $ACR.azurecr.io/agentic-rag-frontend:v1

# Optional: import Chroma so production does not depend on Docker Hub at pull time
az acr import -n $ACR --source docker.io/chromadb/chroma:0.6.3 --image chroma:0.6.3
```

Enable ACR vulnerability scanning (Defender) and lock the registry to the VNet if you use Premium. The Container Apps environment will use a user-assigned identity with AcrPull.

Step 3 — Key Vault and identity

```
az keyvault create -g $RG -n $KV -l $LOC --enable-rbac-authorization true

az identity create -g $RG -n id-rag-apps
PRINCIPAL=$(az identity show -g $RG -n id-rag-apps --query principalId -o tsv)
ID_RES=$(az identity show -g $RG -n id-rag-apps --query id -o tsv)

# Grant the identity read on secrets (Key Vault Secrets User)
az role assignment create --assignee $PRINCIPAL --role "Key Vault Secrets User" \
  --scope $(az keyvault show -n $KV -g $RG --query id -o tsv)

az keyvault secret set --vault-name $KV --name OPENAI-API-KEY --value "sk-..."
az keyvault secret set --vault-name $KV --name API-KEY --value "$API_KEY"
az keyvault secret set --vault-name $KV --name NVIDIA-API-KEY --value "nvapi-..."
az keyvault secret set --vault-name $KV --name REDIS-KEY --value "set-after-redis-exists"
```

Step 4 — Azure Cache for Redis

Create a Standard or Premium C1 (or P1 if you need VNet injection / persistence). Enable TLS. Disable non-SSL port. Prefer a private endpoint on snet-redis. Then write the key into Key Vault and build the URL the API already understands:

```
az redis create -g $RG -n redis-rag-prod -l $LOC --sku Standard --vm-size C1 --minimum-tls-version 1.2

# REDIS_URL example (SSL port 6380):
# rediss://:${REDIS_KEY}@redis-rag-prod.redis.cache.windows.net:6380/0
```

Step 5 — Storage (Blob + Azure Files)

```
az storage account create -g $RG -n stragprod$RANDOM -l $LOC --sku Premium_LRS --kind FileStorage
# Or a Standard_LRS account with both blob and file endpoints for staging

az storage share-rm create --storage-account <account> --name chroma-data --quota 100
az storage share-rm create --storage-account <account> --name app-data --quota 100
az storage container create --account-name <account> --name corpus
az storage blob upload-batch --account-name <account> -d corpus -s data/sample_docs
```

Container Apps mount Azure Files as volumes. Mount chroma-data at `/chroma/chroma` on the Chroma app, and app-data at `/app/data` on the API. Copy PDFs from blob into the file share (AzCopy) before the first ingest job. UID in the API image is 1000 (appuser); set mount options so the process can write parent_store.json and SQLite files.

Step 6 — Container Apps environment

```
az monitor log-analytics workspace create -g $RG -n law-rag -l $LOC
LAW_ID=$(az monitor log-analytics workspace show -g $RG -n law-rag --query customerId -o tsv)
LAW_KEY=$(az monitor log-analytics workspace get-shared-keys -g $RG -n law-rag --query primarySharedKey -o tsv)

az containerapp env create -g $RG -n $ENV -l $LOC \
  --logs-workspace-id $LAW_ID --logs-workspace-key $LAW_KEY \
  --infrastructure-subnet-resource-id <snet-apps-id>
```

Turn on internal load balancer / VNet-only if Front Door is the public edge. Otherwise leave the environment with an external frontend app and internal API/Chroma apps.

Step 7 — Deploy Chroma, API, then frontend

Chroma (min replicas 1, never 0):

```
az containerapp create -g $RG -n chroma --environment $ENV \
  --image $ACR.azurecr.io/chroma:0.6.3 \
  --ingress internal --target-port 8000 \
  --cpu 1.0 --memory 2.0Gi --min-replicas 1 --max-replicas 1 \
  --user-assigned $ID_RES \
  --env-vars IS_PERSISTENT=TRUE ANONYMIZED_TELEMETRY=FALSE
# Attach Azure Files volume chroma-data -> /chroma/chroma
```

API (min replicas 1, 2.0 vCPU / 4Gi):

```
az containerapp create -g $RG -n api --environment $ENV \
  --image $ACR.azurecr.io/agentic-rag-api:v1 \
  --ingress internal --target-port 8000 --transport http \
  --cpu 2.0 --memory 4.0Gi --min-replicas 1 --max-replicas 1 \
  --user-assigned $ID_RES \
  --registry-server $ACR.azurecr.io \
  --secrets openai-key=keyvaultref:https://$KV.vault.azure.net/secrets/OPENAI-API-KEY,identityref:$ID_RES \
            api-key=keyvaultref:https://$KV.vault.azure.net/secrets/API-KEY,identityref:$ID_RES \
            nvidia-key=keyvaultref:https://$KV.vault.azure.net/secrets/NVIDIA-API-KEY,identityref:$ID_RES \
            redis-url=keyvaultref:https://$KV.vault.azure.net/secrets/REDIS-URL,identityref:$ID_RES
```

Non-secret env on the API: `ENVIRONMENT=production`, `CHROMA_MODE=http`, `CHROMA_HOST=<chroma FQDN>`, `CHROMA_PORT=8000`, `CACHE_ENABLED=true`, `RATE_LIMIT_BACKEND=redis`, `API_WORKERS=1`, `TRUST_PROXY_HEADERS=true`, `CORS_ORIGINS=https://$APP_HOST`, `TRUSTED_HOSTS=$APP_HOST`, `INGEST_ALLOWED_ROOTS=/app/data`, `PROTECT_READINESS_ENDPOINT=true`. Probes: liveness HTTP GET /health on 8000. Do not use /health/ready as the platform probe.

Frontend (external ingress, min 2):

```
az containerapp create -g $RG -n frontend --environment $ENV \
  --image $ACR.azurecr.io/agentic-rag-frontend:v1 \
  --ingress external --target-port 80 \
  --cpu 0.25 --memory 0.5Gi --min-replicas 2 --max-replicas 6 \
  --user-assigned $ID_RES \
  --env-vars API_UPSTREAM=http://<api-internal-fqdn>:80 \
  --secrets api-key=keyvaultref:https://$KV.vault.azure.net/secrets/API-KEY,identityref:$ID_RES
# Map secret to env API_KEY. nginx envsubst runs at container start.
```

Internal Container Apps ingress listens on port 80 even when the container target port is 8000. Set `API_UPSTREAM=http://api:80` (or the full internal FQDN with scheme http and no :8000) unless you use TCP ingress. Confirm with `az containerapp show -g $RG -n api --query properties.configuration.ingress`.

Step 8 — Custom domain, TLS, and Front Door

1. On the frontend Container App: add hostname $APP_HOST, create the validation TXT/CNAME.
2. Bind a managed certificate, or use a Key Vault certificate.
3. Set CORS_ORIGINS and TRUSTED_HOSTS to https://$APP_HOST and update the API revision.
4. Optional but recommended: Azure Front Door Premium in front of the frontend FQDN, WAF policy Detection then Prevention, origin timeout ≥ 240–300 seconds for SSE.
5. After TLS is confirmed, uncomment HSTS in frontend/nginx.conf, rebuild, and deploy a new revision.

Step 9 — Ingest

Create a Container Apps Job from the same API image (same files mount, same secrets):

```
az containerapp job create -g $RG -n ingest-job --environment $ENV \
  --image $ACR.azurecr.io/agentic-rag-api:v1 \
  --cpu 2.0 --memory 4.0Gi --replica-timeout 1800 \
  --command python --args -m src.ingestion.ingest --source /app/data/sample_docs

az containerapp job start -g $RG -n ingest-job

curl -sS https://$APP_HOST/api/health
# From an internal debug container, or temporarily:
curl -sS -H "X-API-Key: $API_KEY" https://$APP_HOST/api/health/ready
```

Step 10 — Monitoring

- Container Apps system logs and console logs in the Log Analytics workspace.
- Alert rules: replica restart, 5xx on frontend ingress, Redis server load, Files throttling.
- Application Insights is optional; the API already emits Prometheus on /metrics.
- If you scrape metrics from a sidecar or Grafana agent on the environment, keep the API internal and set PROTECT_METRICS_ENDPOINT=false only for that private scrape path.
- LangSmith remains an external SaaS; store LANGSMITH_API_KEY in Key Vault.

Step 11 — CI/CD

Keep the existing GitHub Actions quality workflow. Add a prod deploy job using `azure/login` with a federated credential (Entra app) — no client secret in GitHub:

1. az acr build or docker buildx push SHA tags to ACR.
2. az containerapp update --image ...:sha for api and frontend.
3. Run ingest-job only when documents or chunking settings changed.
4. Smoke GET /api/health and a canonical query through the public host.

6. Verification

```
curl -i https://rag.example.com/api/health

curl -N https://rag.example.com/api/query/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"What is Self-RAG?","mode":"canonical"}'

# Browser: https://rag.example.com — citations, upload PDF, thumbs feedback.
# Repeat a question and look for cache_hit (Redis).
```

7. Scaling, cost, and limits

Knob	Guidance
Frontend replicas	Autoscale 2–6 on HTTP concurrency
API replicas	Stay at 1; BM25 and semantic cache are in-process
Chroma replicas	1. Files share is the durability layer
Scale to zero	Never on API or Chroma
Front Door origin timeout	Raise for SSE; default is too short for multi-hop
CPU/memory API	2 vCPU / 4Gi starting point; watch working set after ingest

Cost shape: Container Apps (API 2 vCPU always-on) + Redis C1 + Files Premium + NAT + Front Door will usually be smaller than OpenAI token spend. Put a budget on the resource group and a cap on the OpenAI project. Azure OpenAI is an optional later swap for OPENAI_API_KEY / embeddings; it is not required to deploy this repo as-is.

8. Simpler staging path (VM + Compose)

Create an Ubuntu 22.04 VM, install Docker, open 443 via NSG only from your IP or from an Application Gateway, clone the repo, and run the documented compose stack from `docs/PRODUCTION.md`. Point a DNS name at the VM or gateway. Use this only for staging; Redis and Chroma would be on the same disk without backups.

9. Troubleshooting

Symptom	Likely cause	Fix
Revision fails to start	Production config check	Log: API_KEY length, CORS=*, missing OPENAI
UI 502 / 401	API_UPSTREAM host/port wrong or API_KEY mismatch	Internal ingress is port 80; one Key Vault secret
Stream dies at ~60–120s	Front Door / ingress timeout	Raise origin and ACA request timeouts to 320s+
Chroma empty after restart	Files share not mounted	Check volume in container app YAML
Redis connection error	SSL port 6380 / rediss:// and firewall	Private endpoint + REDIS-URL secret
Upload 413	Ingress body limit vs nginx 25m	Raise ACA max request body; INGEST_MAX_UPLOAD_MB
Cold start 20s+	API scaled to 0	min-replicas=1

10. Production checklist

- Key Vault + managed identity; no secrets in ACR images or GitHub variables.
- API and Chroma ingress internal. Redis public access disabled.

- CORS and TRUSTED_HOSTS match https://$APP_HOST.
- API and Chroma minReplicas=1. ALB/Front Door timeout ≥ 300s.
- Ingest job succeeded; /health/ready chroma ok.
- Log Analytics + alerts on 5xx and restarts.
- OpenAI project cap + MAX_TOKENS_PER_HOUR.
- File share snapshots and blob versioning on corpus.
- WAF in detection, then prevention. Budget on the resource group.
- Federated GitHub OIDC deploy; prod branch only.

Related repo docs: `docs/PRODUCTION.md`, `docs/ARCHITECTURE.md`, `docs/GUARDRAILS.md`, `docs/API.md`.